

P5 review

1. P5 的最终定位

P5 不是 recovery planner。P5 只做一件事：

代码块

- 1 判断当前正在执行的轨迹是否还能继续执行。

P5 输出只允许三种 action：

代码块

```
1 enum class IntegrityGateAction {  
2     OK,  
3     REQUEST_REPLAN,  
4     REQUEST_EMERGENCY_STOP  
5 };
```

不要再有：

代码块

```
1 FORWARD_RECOVERY_REPLAN  
2 LATERAL_RECOVERY_REPLAN  
3 BACKTRACK_RECOVERY_REPLAN  
4 REPLAN_MISSION
```

这些 action 看起来更智能，但它们会让 FSM 旁边多出一个隐式 planner，非常难 debug。

恢复方向应该这样处理：

代码块

- 1 普通 REPLAN_TRAJ：
 - 2 仍然调用现有 planFromCurrentTraj()。
 - 3 如果 P1/P2/P3 已经接入，新的轨迹自然会偏向低 PL 区域。
- 4
- 5 EMERGENCY_STOP：
 - 6 只在重规划来不及或重规划失败时触发。

这完全复用你当前 FSM：

代码块

```
1 EXEC_TRAJ -> REPLAN_TRAJ
2 EXEC_TRAJ -> EMERGENCY_STOP
3 EMERGENCY_STOP -> GEN_NEW_TRAJ
```

2. P5 与现有 FSM 的对应关系

你的当前状态机保持不变：

最关键的是：

代码块

```
1 P5 runtime tick 只在 EXEC_TRAJ 中主动触发状态切换。
2 P5 final gate 可在轨迹发布前运行，但不改变 FSM 状态集合。
```

3. P5 两个 hook

最终只需要两个 hook。

Hook A：轨迹发布前 final gate

位置：

代码块

```
1 callReboundReplan()
2 -> updateTrajInfo()
```

3 -> publish planning/bspline 之前

作用：

代码块

1 新轨迹如果明显违反 $PL_{pred} < AL$ ，不发布。

输出：

代码块

```
1    final gate pass:
2       publish bspline, enter EXEC_TRAJ
3
4    final gate fail:
5       planning result considered failed
6       GEN_NEW_TRAJ / REPLAN_TRAJ 保持原状态继续尝试
```

这一步能避免“刚规划出来的轨迹马上被 P5 runtime tick 打回去”。

Hook B：运行时 safety tick

位置：

代码块

1 EGOReplanFSM::checkCollisionCallback()

与 original collision/depth/swarm check 并列。文档中也已经指出 P5 应放在 `checkCollisionCallback()` 附近，作为 $PL_{pred} < AL$ hard check，而默认关闭时不影响原始行为。

运行顺序建议：

代码块

```
1    1. original depth / odom timeout check
2    2. original occupancy / swarm future trajectory check
3    3. P5 integrity trajectory check
4    4. merge action
```

合并规则：

代码块

- 1 EMERGENCY_STOP 优先级最高
- 2 REQUEST_REPLAN 次之
- 3 OK 最低

4. P5 判定逻辑：简单、确定、可 debug

P5 不做复杂分类。只计算五个核心量：

代码块

- 1 current_im_min
- 2 future_min_im
- 3 first_bad_tau
- 4 bad_ratio
- 5 unknown_ratio

然后用固定规则判定。

5. Current gate

输入来自 `/iap/integrity`：

代码块

- 1 current_hpl
- 2 current_vpl
- 3 current_hal
- 4 current_val
- 5 $\text{current_im_h} = \text{current_hal} - \text{current_hpl}$
- 6 $\text{current_im_v} = \text{current_val} - \text{current_vpl}$
- 7 $\text{current_im_min} = \min(\text{current_im_h}, \text{current_im_v})$
- 8 integrity_state
- 9 stamp

判定：

代码块

```
1  current_invalid =
2    /iap/integrity stale
3    OR integrity_state == INVALID
4    OR not finite(current_im_min)
5
6  current_unsafe =
7    current_im_min < current_emergency_margin_m
```

建议默认：

代码块

```
1  p5:
2    current_stale_timeout_s: 0.5
3    current_emergency_margin_m: -0.2
```

解释：

代码块

```
1  current_im_min < 0 不一定马上 EMERGENCY_STOP。
2  只有 current_im_min 明显负到 current_emergency_margin_m 以下，才认为当前完整性已经严重失效。
3
4  轻微负值或接近 0，触发 REPLAN，而不是急停。
```

最终 current gate：

代码块

```
1  if (current_invalid) {
2    return REQUEST_REPLAN;    // 不直接 emergency，除非持续 stale
3  }
4
5  if (current_im_min < current_emergency_margin_m) {
6    return REQUEST_EMERGENCY_STOP;
7  }
8
9  if (current_im_min < current_replan_margin_m) {
10   return REQUEST_REPLAN;
11 }
```

推荐参数：

```
1 current_replan_margin_m: 0.3
2 current_emergency_margin_m: -0.2
```

6. Future trajectory gate

只评估当前执行轨迹：

代码块

```
1 local_data_.position_traj_
2 local_data_.start_time_
3 local_data_.duration_
```

采样窗口：

代码块

```
1 p5:
2     horizon_s: 2.0
3     sample_dt_s: 0.2
```

采样：

代码块

```
1 t_cur = now_s - local_data_.start_time_;
2 t_end = min(local_data_.duration_, t_cur + horizon_s);
3
4 for t in [t_cur, t_end] step sample_dt_s:
5     p = position_traj.evaluateDeBoorT(t)
6     pl = integrity_field.queryPredictedPL(p, now_s + (t - t_cur))
7     al = alert_limit_policy.evaluate(p)
8     im = min(al.hal - pl.hpl, al.val - pl.vpl)
```

每个 sample 只分三类：

代码块

```
1 GOOD:
2     PL valid, AL valid, IM >= future_replan_margin_m
3
4 BAD:
```

```
5    PL valid, AL valid, IM < future_replan_margin_m
6
7    UNKNOWN:
8    PL invalid/stale/unknown OR AL invalid
```

7. Future gate 输出规则

统计：

代码块

```
1    future_min_im
2    first_bad_tau
3    bad_count
4    unknown_count
5    sample_count
6    bad_ratio = bad_count / sample_count
7    unknown_ratio = unknown_count / sample_count
```

最终判定只用四条规则：

代码块

```
1    if (first_bad_tau >= 0.0 && first_bad_tau <= emergency_time_s) {
2        return REQUEST_EMERGENCY_STOP_CANDIDATE;
3    }
4
5    if (future_min_im < future_emergency_margin_m) {
6        return REQUEST_EMERGENCY_STOP_CANDIDATE;
7    }
8
9    if (bad_ratio >= max_bad_ratio) {
10       return REQUEST_REPLAN;
11    }
12
13    if (unknown_ratio >= max_unknown_ratio) {
14       return REQUEST_REPLAN;
15    }
16
17    return OK;
```

但是注意：P5 不应该直接把 REQUEST_EMERGENCY_STOP_CANDIDATE 变成 EMERGENCY_STOP。

它应该复用 original EGO 的逻辑：

代码块

```
1  若违例临近：
2    先尝试 planFromCurrentTraj()
3    如果重规划成功，继续 EXEC_TRAJ
4    如果重规划失败，才进入 EMERGENCY_STOP
```

也就是说最终 merge 逻辑为：

代码块

```
1  if (p5_action == OK) {
2    keep EXEC_TRAJ;
3  }
4
5  if (p5_action == REQUEST_REPLAN) {
6    changeFSMExecState(REPLAN_TRAJ, "P5");
7  }
8
9  if (p5_action == REQUEST_EMERGENCY_STOP_CANDIDATE) {
10   bool replan_success = planFromCurrentTraj();
11   if (!replan_success) {
12     callEmergencyStop(odom_pos_);
13     changeFSMExecState(EMERGENCY_STOP, "P5");
14   }
15 }
```

这和 original collision check 的设计对齐：原始逻辑也是发现未来碰撞后先尝试 `planFromCurrentTraj()`，失败且临近才急停。

8. 推荐 P5 参数

最终参数不要太多：

代码块

```
1  p5:
2    enable: true
3
4    horizon_s: 2.0
5    sample_dt_s: 0.2
6
```



```
7    current_stale_timeout_s: 0.5
8    field_stale_timeout_s: 0.7
9
10   current_replan_margin_m: 0.3
11   current_emergency_margin_m: -0.2
12
13   future_replan_margin_m: 0.3
14   future_emergency_margin_m: -0.5
15
16   max_bad_ratio: 0.25
17   max_unknown_ratio: 0.30
18
19   bad_tick_to_replan: 2
20   good_tick_to_clear: 2
21
22   log_samples: true
```

不要再加：

代码块

```
1    forward_search_radius
2    lateral_search_radius
3    backtrack_timeout
4    last_safe_pose_distance
5    recovery_target_validity
```

这些参数会显著增加 debug 难度，而且会把 P5 变成一个隐式 planner。

9. P5 debouncing：避免抖动

为避免单帧 predictor 抖动导致频繁 `EXEC_TRAJ -> REPLAN_TRAJ`，P5 只保留两个计数器：

代码块

```
1    int bad_tick_count = 0;
2    int good_tick_count = 0;
```

逻辑：

代码块

```
1    if (raw_action == OK) {
2        good_tick_count++;
```

```

3     bad_tick_count = 0;
4 } else {
5     bad_tick_count++;
6     good_tick_count = 0;
7 }
8
9 if (raw_action == REQUEST_REPLAN && bad_tick_count < bad_tick_to_replan) {
10     return OK_WITH_WARNING;
11 }
12
13 if (raw_action == REQUEST_EMERGENCY_STOP_CANDIDATE) {
14     return REQUEST_EMERGENCY_STOP_CANDIDATE; // 不 debounce
15 }

```

这样：

代码块

- 1 非临近风险：需要连续 2 tick 才 replan。
- 2 临近风险：立即处理。

这比 consecutive unsafe sample、unsafe ratio、severity、forward recovery 混在一起更好 debug。

10. P5 debug 输出

必须发布一个轻量 status：

代码块

```
1 /iap/planner_integrity_status
```

字段建议：

代码块

```

1 stamp
2 fsm_state
3 action
4 raw_action
5 reason
6
7 current_im_min

```

```
8   future_min_im
9   first_bad_tau
10  bad_ratio
11  unknown_ratio
12
13  sample_count
14  bad_count
15  unknown_count
16
17  field_generation_id
18  field_age_s
19  current_integrity_age_s
```

debug 时你只需要看：

代码块

```
1   为什么触发？
2       reason
3
4   是当前触发还是未来触发？
5       current_im_min / future_min_im
6
7   还有多久出问题？
8       first_bad_tau
9
10  是不是 unknown 太多？
11      unknown_ratio
12
13  是不是 stale？
14      field_age_s / current_integrity_age_s
```

这样比 forward/lateral/backtrack recovery 的状态机容易排查很多。

最终 P5 伪代码

代码块

```
1   IntegrityGateResult PlannerIntegritySupervisor::check(
2       const EGOReplanFSM& fsm,
3       const LocalTrajData& local_data,
4       double now_s) {
5
```

```
6 IntegrityGateResult result;
7 result.action = IntegrityGateAction::OK;
8
9 if (!enable_) {
10     return result;
11 }
12
13 if (fsm.exec_state_ != EXEC_TRAJ) {
14     result.reason = "not_in_exec_traj";
15     return result;
16 }
17
18 // ----- 1. current gate -----
19 auto cur = current_integrity_buffer_.latest();
20 if (!cur.fresh(now_s, params_.current_stale_timeout_s)) {
21     result.raw_action = IntegrityGateAction::REQUEST_REPLAN;
22     result.reason = "current_integrity_stale";
23     return debounce(result);
24 }
25
26 double current_im_min = std::min(cur.hal - cur.hpl, cur.val - cur.vpl);
27 result.current_im_min = current_im_min;
28
29 if (!std::isfinite(current_im_min)) {
30     result.raw_action = IntegrityGateAction::REQUEST_REPLAN;
31     result.reason = "current_integrity_invalid";
32     return debounce(result);
33 }
34
35 if (current_im_min < params_.current_emergency_margin_m) {
36     result.raw_action = IntegrityGateAction::REQUEST_EMERGENCY_STOP_CANDIDATE;
37     result.reason = "current_im_severely_negative";
38     return result;
39 }
40
41 if (current_im_min < params_.current_replan_margin_m) {
42     result.raw_action = IntegrityGateAction::REQUEST_REPLAN;
43     result.reason = "current_im_below_replan_margin";
44     return debounce(result);
45 }
46
47 // ----- 2. future gate -----
48 if (!local_data.hasTrajectory()) {
49     result.reason = "no_local_trajectory";
50     return result;
51 }
52
```

```

53     TrajectoryIntegrityReport rep =
54         integrity_field_.evaluateTrajectoryCached(
55             local_data.position_traj_,
56             local_data.start_time_,
57             now_s,
58             alert_limit_policy_);
59
60     result.future_min_im = rep.min_im;
61     result.first_bad_tau = rep.first_bad_tau;
62     result.bad_ratio = rep.bad_ratio;
63     result.unknown_ratio = rep.unknown_ratio;
64     result.field_generation_id = rep.generation_id;
65     result.field_age_s = rep.field_age_s;
66
67     if (rep.first_bad_tau >= 0.0 &&
68         rep.first_bad_tau <= params_.emergency_time_s) {
69         result.raw_action = IntegrityGateAction::REQUEST_EMERGENCY_STOP_CANDIDATE;
70         result.reason = "future_violation_imminent";
71         return result;
72     }
73
74     if (rep.min_im < params_.future_emergency_margin_m) {
75         result.raw_action = IntegrityGateAction::REQUEST_EMERGENCY_STOP_CANDIDATE;
76         result.reason = "future_im_severely_negative";
77         return result;
78     }
79
80     if (rep.bad_ratio >= params_.max_bad_ratio) {
81         result.raw_action = IntegrityGateAction::REQUEST_REPLAN;
82         result.reason = "future_bad_ratio_high";
83         return debounce(result);
84     }
85
86     if (rep.unknown_ratio >= params_.max_unknown_ratio) {
87         result.raw_action = IntegrityGateAction::REQUEST_REPLAN;
88         result.reason = "future_unknown_ratio_high";
89         return debounce(result);
90     }
91
92     result.raw_action = IntegrityGateAction::OK;
93     result.action = IntegrityGateAction::OK;
94     result.reason = "integrity_ok";
95     return result;
96 }

```

FSM 侧处理:

代码块

```
1  auto p5 = integrity_supervisor_->check(*this, planner_manager_->local_data_,
    now_s);
2
3  if (p5.action == IntegrityGateAction::OK) {
4      return;
5  }
6
7  if (p5.action == IntegrityGateAction::REQUEST_REPLAN) {
8      changeFSMExecState(REPLAN_TRAJ, "P5_REPLAN");
9      return;
10 }
11
12 if (p5.action == IntegrityGateAction::REQUEST_EMERGENCY_STOP_CANDIDATE) {
13     bool success = planFromCurrentTraj();
14     if (success) {
15         changeFSMExecState(EXEC_TRAJ, "P5_REPLAN_SUCCESS");
16     } else {
17         callEmergencyStop(odom_pos_);
18         changeFSMExecState(EMERGENCY_STOP, "P5_EMERGENCY");
19     }
20     return;
21 }
```

需要从当前文档中删除或降级的内容

你现在第 11 章的这些内容建议删除：

代码块

```
1  FORWARD_RECOVERY_REPLAN
2  LATERAL_RECOVERY_REPLAN
3  BACKTRACK_RECOVERY_REPLAN
4  last_safe_pose
5  searchForwardRecoveryTarget()
6  searchLateralRecoveryTarget()
7  validateBacktrackTarget()
```

原因：

代码块

- 1 1. 它们不属于 supervisor，而属于 planner / front-end / recovery planner。
- 2 2. 会引入额外目标选择逻辑，和 getLocalTarget() / planFromCurrentTraj() 重叠。
- 3 3. 会让 debug 从“为什么触发 replan”变成“为什么选这个 recovery target”，复杂度翻倍。
- 4 4. 和现有 FSM 状态不对齐。

可以保留一句作为未来扩展：

代码块

- 1 Future extension:
- 2 若普通 REPLAN_TRAJ 多次失败，可由 P3/P4 生成 recovery-biased local target。
- 3 该功能不属于 P5 v1/v2 的核心职责。